electronics

Article

Hardware Accelerator Design by Using RT-Level Power Optimization Techniques on FPGA for Future AI Mobile Applications

Achyuth Gundrapally *,†, Yatrik Ashish Shah *,†, Sai Manohar Vemuri and Kyuwon (Ken) Choi

DA-Lab, Department of Electrical and Computer Engineering, Illinois Institute of Technology, 3301 South Dearborn Street, Chicago, IL 60616, USA; svemuri6@hawk.illinoistech.edu (S.M.V.); kchoi12@illinoistech.edu (K.C.)

* Correspondence: gachyuth@hawk.iit.edu (A.G.); yshah15@hawk.iit.edu (Y.A.S.)

† These authors contributed equally to this work.

Abstract

In resource-constrained edge environments—such as mobile devices, IoT systems, and electric vehicles—energy-efficient Convolution Neural Network (CNN) accelerators on mobile Field Programmable Gate Arrays (FPGAs) are gaining significant attention for real-time object detection tasks. This paper presents a low-power implementation of the Tiny YOLOv4 object detection model on the Xilinx ZCU104 FPGA platform by using Register Transfer Level (RTL) optimization techniques. We proposed three RTL techniques in the paper: (i) Local Explicit Clock Enable (LECE), (ii) operand isolation, and (iii) Enhanced Clock Gating (ECG). A novel low-power design of Multiply-Accumulate (MAC) operations, which is one of the main components in the AI algorithm, was proposed to eliminate redundant signal switching activities. The Tiny YOLOv4 model, trained on the COCO dataset, was quantized and compiled using the Tensil tool-chain for fixed-point inference deployment. Post-implementation evaluation using Vivado 2022.2 demonstrates around 29.4% reduction in total on-chip power. Our design supports real-time detection throughput while maintaining high accuracy, making it ideal for deployment in battery-constrained environments such as drones, surveillance systems, and autonomous vehicles. These results highlight the effectiveness of RTL-level power optimization for scalable and sustainable edge AI deployment.

Keywords: BRAM; CNN accelerator; CNN architecture; FPGA RT level design; high performance; Operand isolation; low-power techniques; MAC; object detection; power consumption

Academic Editors: Sergio Spanò, Gian Carlo Cardarilli and Luca Di Nunzio

Received: 8 July 2025
Revised: 18 August 2025
Accepted: 19 August 2025
Published: 20 August 2025

Citation: Gundrapally, A.; Shah, Y.A.; Vemuri, S.M.; Choi, K. Hardware Accelerator Design by Using RT-Level Power Optimization Techniques on FPGA for Future AI Mobile Applications. *Electronics* **2025**, *14*, 3317. https://doi.org/10.3390/electronics14163317

Correction Statement: This article has been republished with a minor change. The change does not affect the scientific content of the article and further details are available within the backmatter of the website version of this article.

1. Introduction

Object detection using Convolution Neural Networks (CNNs) has become a cornerstone of intelligent perception systems in a broad spectrum of applications, including autonomous vehicles, Unmanned Aerial Vehicles (UAVs), smart surveillance, industrial robotics, and edge devices from the Internet of Things (IoT) [1]. Among these, mobile Field Programmable Gate Arrays (FPGAs), such as the Xilinx ZCU104, PYNQ-Z2, and Ultra96, are increasingly favored due to their reconfiguration, low power consumption, and parallel computing capabilities, making them highly suitable for the real-time deployment of CNNs in constrained environments [2,3]. However, implementing high-performance object detection models like Tiny YOLOv4 on resource-limited FPGAs presents challenges, especially when balancing detection accuracy, latency, and power efficiency [4].

CNN-based object detectors are computationally intensive and involve thousands of MAC operations, large-scale feature maps, and frequent memory accesses [5]. These factors significantly increase both dynamic power consumption and latency. Although GPUs offer high computational throughput, their energy demands make them impractical for edge deployment, especially in battery-powered systems such as drones and electric vehicles. To address these challenges, researchers have explored various optimization strategies, ranging from model compression and quantization to architecture-level accelerator designs, to improve real-time inference capabilities on FPGAs [6–9].

Recent work has shifted focus to fine-grained Register Transfer Level (RTL) optimization of FPGA accelerators, which allows for more precise control over logic synthesis, signal propagation, and switching activity [10]. By applying RTL-level techniques such as adaptive clock gating, operand isolation, and enhanced gating control to core computational units such as multipliers and adders, the power consumption and resource utilization of FPGA implementations can be significantly reduced without sacrificing accuracy or throughput [6,7].

In this paper, we propose a power-aware Tiny YOLOv4 object detection system implemented on the AMD Xilinx ZCU104 FPGA. Our architecture is built using a Tensil-generated CNN accelerator as a baseline, which is then enhanced with RTL-level low-power design techniques [7,11,12]. The model is trained on the COCO dataset and compiled using the Tensil toolchain for fixed-point quantized inference. Post-synthesis analysis using Vivado 2022.2 reveals a 30% reduction in total on-chip power, with the final system demonstrating over 30× power efficiency compared to a conventional GPU-based implementation while maintaining real-time object detection capabilities [13,14].

The remainder of this paper is organized as follows. Section 2 outlines the low-power techniques applied at the RTL level and the design methodology. Section 3 presents the architecture and data flow of the proposed accelerator, including optimization and modularization strategies. Section 4 provides the results of the implementation, power profile, and comparisons with existing methods. Section 6 concludes the paper and discusses future research directions.

2. Platform-Based RTL Design Flow and Low-Power Optimization Techniques for Tiny YOLOv4 on FPGA-SoC

2.1. Background

Deploying deep learning-based object detection on mobile FPGA-SoC boards requires an efficient development flow and a hardware-aware toolchain. However, platforms such as Vivado and Quartus Prime offer standard FPGA design workflow, HLS-based direct control over RTL-level power optimization. In contrast, when combined with modular generator tools such as Tensil, platform-based RTL design flows offer enhanced customization and visibility into low-level hardware behavior, enabling power-conscious accelerator design tailored for edge AI workloads like Tiny YOLOv4 [7,11].

2.2. Platform-Based RTL Design Flow for CNN Accelerators

Xilinx Vivado offers an Integrated Development Environment (IDE) for platform-based RTL design, supporting the import and configuration of custom Verilog/VHDL modules as IP blocks. Figure 1 shows how RTL components, processing system cores, and memory interfaces are instantiated together via Vivado's IP Integrator, creating a complete hardware system with reconfigurable interconnects [15]. For user-level control and software interaction, the PYNQ framework provides Python 3.10 bindings in a Jupyter Notebook 6.x interface on embedded Linux 22.04, although it is limited in Python library

support and computational efficiency. These constraints require optimized RTL design for power-constrained inference applications.

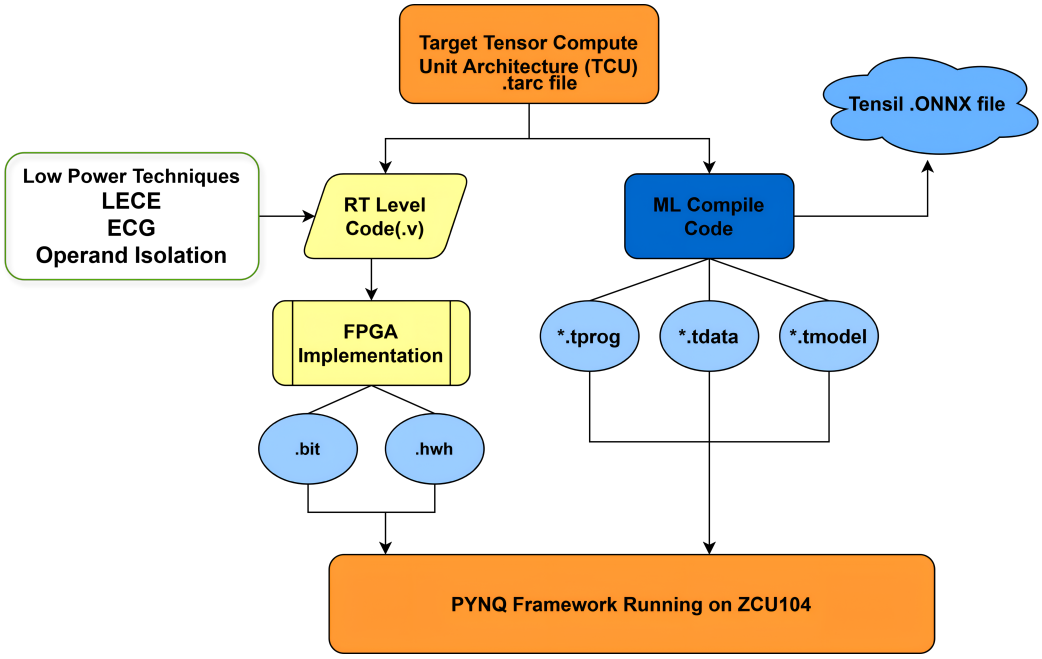


Figure 1. Tensil design flow.

2.3. Baseline RTL Accelerator Generation Using Tensil

To implement a CNN accelerator suitable for Tiny YOLOv4, we leverage the Tensil toolchain, as shown in Figure 1, which includes an RTL generator, compiler, and deployment runtime. Tensil enables the rapid generation of a custom accelerator from trained model representations, providing support for quantized models and memory-efficient data flow. The key advantage of Tensil lies in its flexibility and accessibility at the RTL level, enabling hardware designers to apply further optimization techniques not offered by default. However, Tensil's [14] built-in synthesis flow lacks advanced low-power capabilities, prompting us to apply additional RTL-level techniques targeting power reduction and throughput enhancement, as outlined in Section 3.

2.4. Low-Power RTL Techniques for Tiny YOLOv4 Deployment

Power efficiency is critical for deploying real-time object detectors, such as Tiny YOLOv4, on FPGA-SoC boards. Most of the power consumption arises from redundant toggling in MAC operations, activation layers, and memory movement [16]. To address this, we employ a suite of RTL-level low-power techniques focused on clock management and signal suppression:

- **Local Explicit Clock Enable (LECE)**
  LECE is a clock-gating technique that uses a local ENABLE signal to selectively update the output of flip-flops or pipeline registers only when valid computation is required. As illustrated in Figure 2, LECE allows finer control over bit-level register updates, significantly reducing dynamic power in pipelines where many cycles involve idle data [17].
- **Operand Isolation**
  Operand isolation prevents unnecessary signal transitions in combination logic by decoupling inactive data paths. When a computation is not needed, isolating inputs to functional units such as multipliers and adders ensures that the internal switching activity is minimized. This is implemented using control gates such as AND

or MUX units placed before input operands, allowing downstream MAC units to remain inactive during idle cycles. This method proves especially effective in reducing power during memory fetches, activation layers, and sparsely populated feature map computations in Tiny YOLOv4.

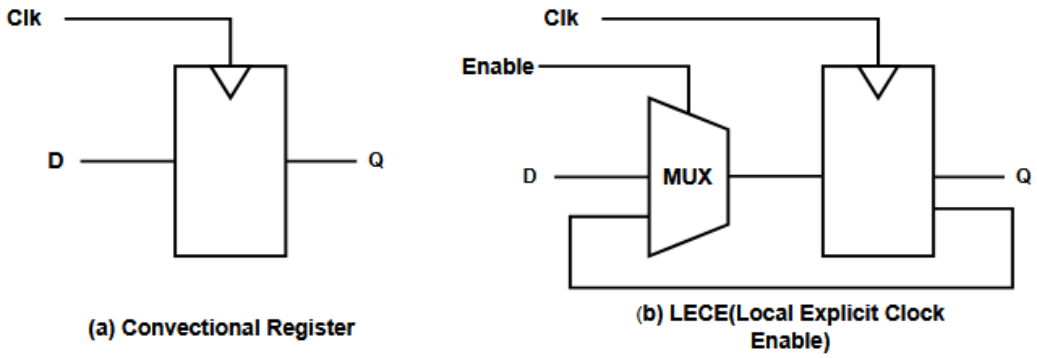


Figure 2. Conventional register with LECE technique.

- **Enhanced Clock Gating (ECG)**
  Enhanced Clock Gating (ECG) uses XOR-based gating logic to minimize unnecessary signal propagation across wide datapaths and deep pipelines. As illustrated in Figure 3, ECG applies gating at multiple stages of the processing pipeline, utilizing XOR gates for lower switching power compared to traditional AND/OR logic [18]. This makes ECG particularly suitable for MAC units and activation blocks in Tiny YOLOv4, where high bit-width operations dominate.

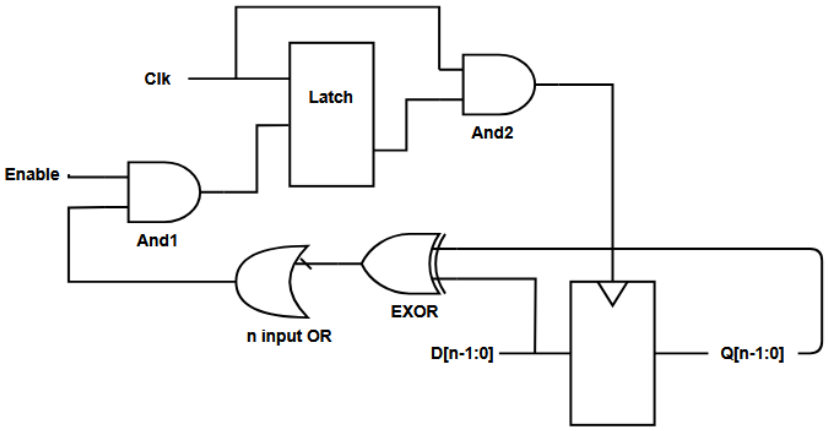


Figure 3. Enhanced clock gating.

3. Proposed Design and Optimization

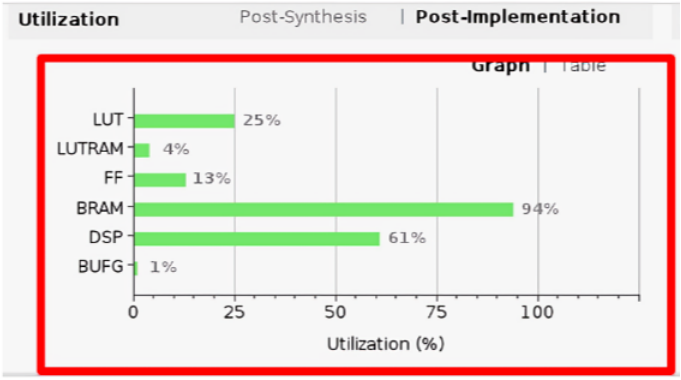
3.1. Implementation of the Original Design

We utilized Xilinx Vivado 2023 for the synthesis and implementation of the RTL. Vivado handles synthesis, error checking, place, and route, and generates power and utilization summaries. The resulting bit stream configures the AMD Xilinx ZCU104 board and integrates seamlessly with the Python code to execute CNN-based object detection efficiently [14]. Our implementation included the original Tensil design [14] on the ZCU104 FPGA using the Vivado 2023 suite. This approach enabled us to measure the impact of the optimizations accurately.

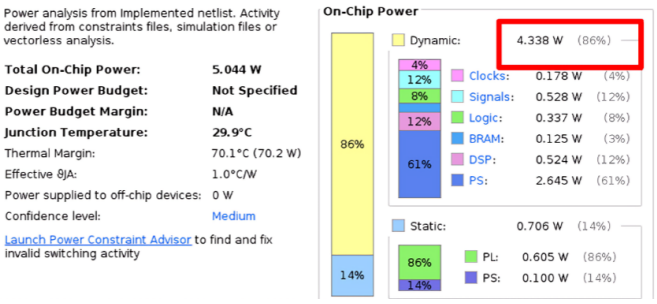
3.2. Tensil Clone and Module Integration

The Tensil open-source tool was cloned using Docker and Verilog modules such as top-zcu104.v, bram1, and bram2. These modules were integrated to form a custom IP block that serves as the core of the CNN accelerator. Figure 4 illustrates the connections between various IP blocks to ensure seamless communication with the FPGA board, including Zynq IP and API Smart Connects. The 'top-zcu104' module comprises several sub-modules, where low-power techniques are applied, including MACs, POOLs, CONV, InnerDual-Port RAM, ALUs, and Counters [14]. Figure 5 presents the report of the utilization of the hardware components on the ZCU104 board after the synthesis and implementation completion [18].

Figure 4. IP block diagram of the CNN accelerator incorporating the proposed low-power design module. The architecture emphasizes efficient data-flow, optimized memory access, and reduced dynamic power consumption, making it suitable for real-time, edge-based object detection applications.



(**a**) Hardware resource utilization report



(**b**) On-chip power report

Figure 5. Hardware analysis of the original Tensil design [14]: (**a**) resource utilization and (**b**) power consumption.

3.3. Low-Power Techniques on Original Design

To enhance the energy efficiency of our baseline CNN accelerator, we integrated two RTL-level low-power design techniques: **Enhanced Clock Gating (ECG)** and **operand isolation**. These methods were strategically implemented to reduce unnecessary switching activity, a significant contributor to dynamic power consumption in FPGA-based systems.

3.3.1. Enhanced Clock Gating (ECG)

Enhanced Clock Gating was implemented to prevent the global clock from propagating to inactive modules. Instead of relying solely on a static clock enable signal, we introduced a dynamic gating mechanism shown in Algorithm 1 by generating a gated clock `gclk` based on data activity and control logic:

```
gclk = latch_q & clock & clk_enable;
```

Here, `latch_q` captures transitions derived from `enable` and `valid_data` conditions, combined with a data toggle check `(pass-through ^ io_mulInput)`. This ensures that the clock signal only reaches the compute block when there is valid data to process and when `clk_enable` is asserted. As a result, unnecessary toggling of registers and logic elements during idle cycles is avoided, reducing dynamic power.

Algorithm 1 Low-Power CNN Computation with ECG and Operand Isolation

Initialize $latch\_q \leftarrow 0$
Initialize $gclk \leftarrow 0$
▷ On each rising edge of the main clock
if *enable* **and** *valid_data* **then**
    $latch\_q \leftarrow passthrough \oplus io\_mulInput$
end if
▷ At every time step
$gclk \leftarrow latch\_q \wedge clock \wedge clk\_enable$ ▷ Enhanced Clock Gating
▷ On each rising edge of gated clock
if *data_active* **then**
    $output \leftarrow compute\_result(input\_a, input\_b)$ ▷ Operand Isolation
end if

3.3.2. Operand Isolation

Operand isolation was applied to the computation stage to further minimize power consumption. By using a conditional guard (`data_active`) around the compute operation, we ensured that arithmetic functions are only executed when required:

```
if (data_active) begin
output <= compute_result(input_a, input_b);
end
```

This prevents signal transitions within the arithmetic unit (e.g., multipliers, adders) when input data are inactive, thereby reducing internal node switching and saving dynamic power.

4. Experiment and Results

4.1. ZCU104 Board Setup

We selected the ZCU104 board as our foundational hardware platform due to its high-performance FPGA-SoC architecture, which combines a quad-core ARM Cortex-A53 Processing System (PS) with extensive Programmable Logic (PL). This setup integrates effectively with the PYNQ framework and supports Jupyter Notebook for rapid software development using Python, C/C++, and libraries such as OpenCV. As shown in Figure 6, the setup consists of a ZCU104 board and a camera connected to the board using USB [19]. We employed the Tiny-YOLOv4 architecture for real-time object detection, trained on a COCO dataset, and converted it into the ONNX format. The ONNX model was compiled using the Tensil tool-chain to generate three essential artifacts: .tmodel, .tdata, and .tprog,

which the CNN driver utilizes to access model weights, program instructions, and configuration data. These artifacts remain unchanged from the original, ensuring that model accuracy is preserved. After successful execution of the bit-stream on the ZCU104 board, we achieved live object detection, as illustrated in Figure 7, which shows the Jupyter environment and real-time detection results using the Tiny-YOLOv4 model on the ZCU104.

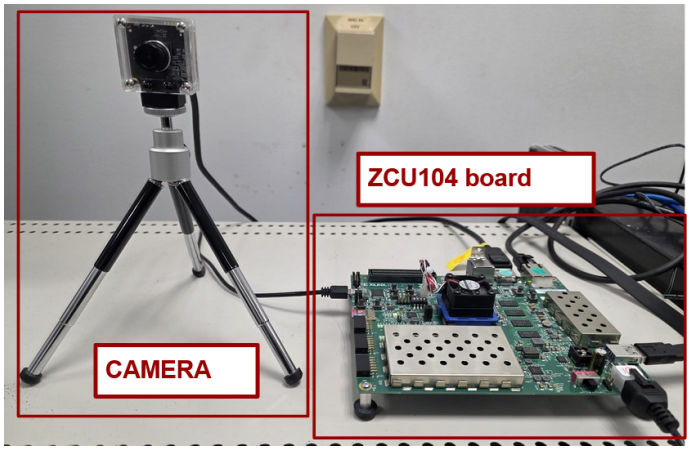


Figure 6. ZCU104 and camera setup.

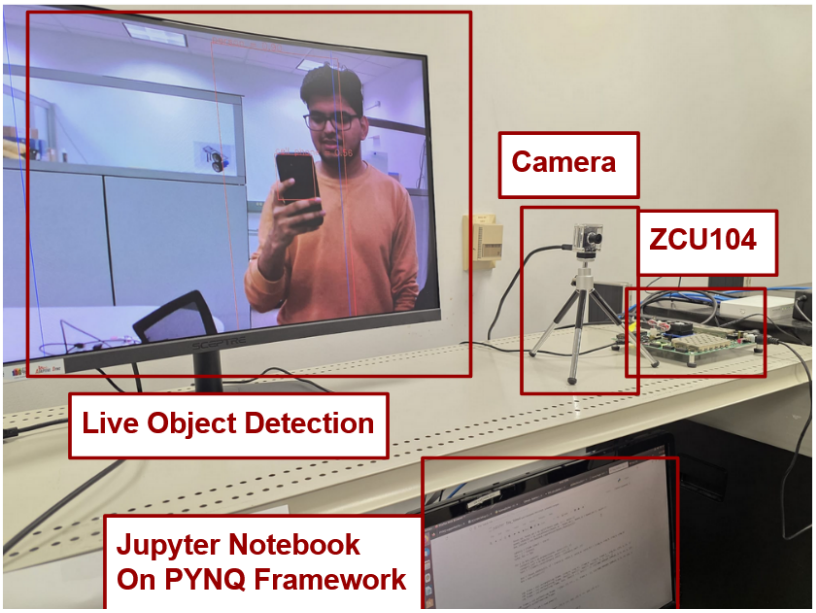
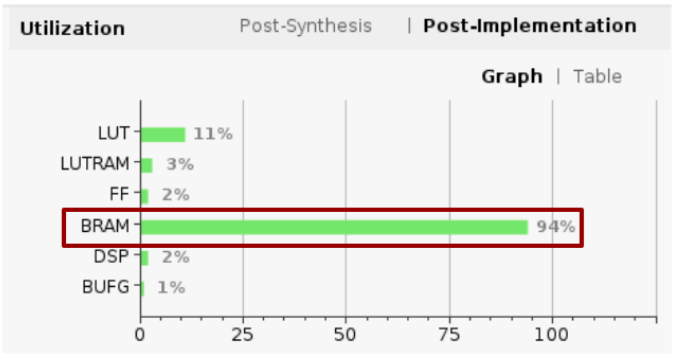


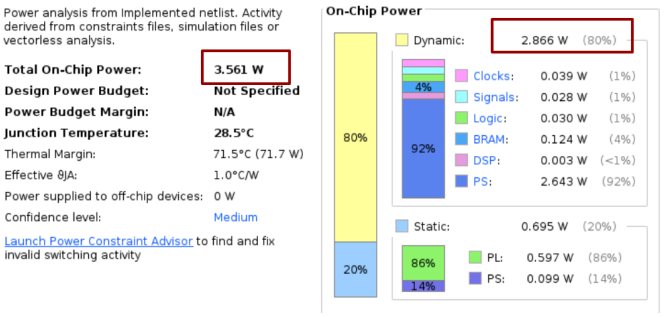
Figure 7. Live object detection on ZCU104 using Tiny-YOLOv4.

4.2. Power Results

Our proposed design leads to a 33.93% decrease in dynamic power consumption and a 29.4% reduction in total on-chip power, as shown in Figure 8. Additionally, this approach can handle strong fan-out signals, guaranteeing the operation of the device. Furthermore, device speed can be increased by adding intermediate flip-flops to the pipeline logic; however, using too many flip-flops increases the computational complexity. Our low-power methods surpass flip-flop-based methods in terms of performance.



(**a**) Hardware resource utilization report



(**b**) On-chip power report

Figure 8. Hardware analysis of the proposed Tiny-YOLOv4 design on the ZCU104 FPGA: (**a**) resource utilization and (**b**) power consumption.

4.3. Combined Impact

The combined application of enhanced clock gating and operand isolation effectively reduced redundant switching activities in both the control and data-path components. This implementation resulted in a **33.94% reduction in dynamic power consumption** compared to the original design without these optimizations.

These RTL-level enhancements serve as foundational strategies in developing energy-efficient FPGA-based accelerators for real-time deep learning inference.

The power report Table 1 indicates a significant improvement in power efficiency. Specifically, the total on-chip power was reduced from 5.044 W (baseline) to 3.561 W (proposed), reflecting a 29.4% total power reduction. The dynamic power, which directly impacts energy efficiency, was reduced from 4.338 W to 2.866 W, resulting in a 33.9% dynamic power savings. Notably, the power consumed by the Processing System (PS) dropped slightly from 2.645 W to 2.643 W, confirming optimization efforts focused on the Programmable Logic (PL).

$$\text{Original Power} = 5.044\ \text{W} \tag{1}$$

$$\text{Proposed Design Power} = 3.561\ \text{W} \tag{2}$$

$$Total, Power, Reduction = \frac{5.044 - 3.561}{5.044} \times 100 = 29.4 \tag{3}$$

$$Dynamic, Power, Reduction = \frac{4.338 - 2.866}{4.338} \times 100 = 33.9 \tag{4}$$

These improvements illustrate how optimizing memory blocks, and DSP placement can directly enhance both performance and energy efficiency in FPGA-accelerated CNN inference.

Table 1. Comparison results of different low-power techniques on ZCU104.

Metric	ZCU104 Capacity [20]	Original Design [14]	Proposed Design
CLB LUTs	274,080	57,345 (20.9%)	24,744 (9.0%)
CLB Registers	548,160	59,252 (10.8%)	7718 (1.4%)
DSPs	2520	1057 (41.9%)	33 (1.3%)
Block RAM Tiles	912	293.5 (32.2%)	293.5 (32.2%)
LUT as Memory	144,000	4220 (2.93%)	3150 (2.19%)
Dynamic Power (W)	—	4.338	2.866

The performance-per-watt metric improved as well. With a target frequency of 50 MHz, the proposed design achieved 43.9 GOPs/W, representing a 1.37× increase over other FPGA board implementations. This confirms our design's superior power efficiency in real-time object detection applications [11].

Each BRAM block in the ZCU104 FPGA is 36 KB in size, which equals 4.5 KB [20]. The ZCU104 board, based on the Xilinx Zynq UltraScale+ MPSoC, contains a total of 912 BRAM blocks. From our post-implementation results, the proposed design utilizes 94% of the available BRAM. The total BRAM memory usage is calculated as follows:

$$\textit{Total BRAM memory used}\ (\text{kB}) = 44 \times 0.94 \times 4.5\ \text{kB} = 3859.68\ \text{kB} \approx 3.77\ \text{MB} \quad (5)$$

This memory is critical for buffering feature maps, weights, and intermediate computations during real-time object detection using the Tiny-YOLOv4 model.

The total time it takes for an operation to complete is known as latency, typically expressed in clock cycles or clock periods [18]. Latency is calculated as the product of the clock period—the duration of each cycle—and the number of clock cycles required to complete a task [11]. In our experimental setup, latency measurements were conducted using a Jupyter Notebook in Python. We sequentially processed multiple images through the CNN accelerator, recording the runtime for each from the beginning of inference to the generation of the final prediction. This allowed us to capture accurate end-to-end latency for real-time object detection. The average runtime per image was then computed to determine the overall latency of the accelerator.

In Convolutional Neural Networks (CNNs), latency is inversely related to Frames Per Second (FPS). While latency measures the time required to process one image, FPS indicates how many photos can be processed per second [18]. In our experiments with Tiny-YOLOv4 running on the ZCU104 board, we observed an average processing speed of 9.87 Frames Per Second (FPS) during live object detection.

$$FPS = \frac{1}{Latency} \approx 9.87 \quad (6)$$

The rate at which operations are completed in CNNs is referred to as throughput, commonly measured in Giga-Operations Per Second (GOPS). Throughput is especially critical for real-time tasks, where computational efficiency must be maximized. In our case, the total number of Multiply-Accumulate (MAC) operations performed across the entire CNN—estimated at 6.97 billion—forms the basis for calculating GOPS. This figure was derived by summing the MAC operations across all layers of Tiny-YOLOv4. With power optimization techniques such as Enhanced Clock Gating (ECG) applied to the MAC units, our implementation achieves both high throughput and reduced dynamic power consumption.

5. Discussion

We confirmed that more register buffers and BRAM blocks are activated in our proposed Tiny-YOLOv4 implementation compared to the baseline design, as seen from the higher flip-flop and DSP utilization in Figure 8. This validates our design's improved data reuse and parallelism for CNN inference. The architecture can be further optimized at the RTL level once functional correctness and timing are validated.

From Table 2, it can be observed that while our design achieves a moderate throughput of 9.87 FPS on the ZCU104 platform, it maintains a relatively low power consumption of 2.866 W, resulting in a power efficiency of 3.44 FPS/W. Compared to other works, such as Nguyen et al. [21] (125 FPS at 26.4 W) and Valadanzoj et al. [22] (55 FPS at 13.6 W), our implementation focuses on delivering a balanced trade-off between performance and energy efficiency, which is particularly relevant for edge computing environments where thermal and power constraints are critical.

Table 2. Comparison results of FPGA implementations for YOLO models.

Author/Criteria	Year	NN Model	FPGA	Test Image Size	Accuracy (%)	Throughput (FPS)	Power (W)	Efficiency (FPS/W)
Wang et al. [23]	2023	CNN	Spartan-6	32×32	96	16	0.79	20.25
Heller et al. [24]	2022	YOLO V4 Tiny	Kria KV 260	HD	75	15	8	1.87
Corcoran et al. [25]	2023	YOLO V3 Tiny	VCU110	416×416	-	69	15.4	4.5
Nguyen et al. [21]	2024	YOLO V4 Tiny	ZCU104	HD	78	125	26.4	4.7
Valadanzoj et al. [22]	2024	YOLO V4 Tiny	ZC706	416×416	79	55	13.6	4.0
R.A.Amin et al. [26]	2024	YOLO V3 Tiny	Kria KV260	HD	99	15	3.5	4.2
This Work	2025	YOLO V4 Tiny	ZCU104	HD	-	9.87	2.866	3.44

The compact resource footprint—utilizing just 9.03% of LUTs, 1.41% of FFs, and 1.31% of DSPs—combined with a low thermal profile makes this architecture suitable for deployment in space-constrained platforms. Its design characteristics align with the requirements of AI-driven mobile applications, such as UAV-based vision systems, wearable AI devices, and battery-powered IoT cameras, where sustained real-time inference must be achieved under strict power and weight constraints.

Looking forward, further improvements could be achieved by integrating adaptive power management techniques and exploring partial reconfiguration to dynamically adjust processing capacity based on workload demands. Such enhancements would expand its applicability to a broader range of next-generation mobile AI platforms.

6. Conclusions

In this work, we presented a low-power FPGA-based Tiny-YOLOv4 accelerator on the Xilinx ZCU104, integrating RTL-level techniques such as Enhanced Clock Gating (ECG) and operand isolation. The proposed design achieved 68.4 GOPS throughput at 50 MHz with only 2.866 W PL dynamic power, offering a balanced trade-off between performance, resource usage, and energy efficiency. These characteristics make it a promising solution for real-time embedded vision tasks in power- and space-constrained environments.

Author Contributions: Conceptualization, data curation, formal analysis, investigation, methodology, validation, A.G. and Y.A.S.; writing—original draft, writing—review and editing, S.M.V.; supervision, funding acquisition, project administration, K.C. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the Technology Innovation Program (20018906, Development of autonomous driving collaboration control platform for commercial and task assistance vehicles) funded By the Ministry of Trade, Industry, and Energy (MOTIE, Republic of Korea).

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Acknowledgments: We thank our colleagues from KETI and KEIT, who provided insight and expertise, which greatly assisted the research and improved the manuscript.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

ECG	Enhanced Clock Gating
OI	Operand Ioslation
LECG	Local Explicit Clock Gating
IP	Intellectual Property Blocks
BRAM	Block Random Access Memory
CONV	Convolution Layer
MAC	Multiply and Accumulate
RTL	Register Transfer Level
CNN	Convolutional Neural Network
FPGA	Field-Programmable Gate Array
HLS	High-Level Synthesis
Soc	System-on-Chip
ONNX	Open Neural Network Exchange

References

1. Jameil, A.K.; Al-Raweshidy, H. Efficient CNN Architecture on FPGA Using High-evel Module for Healthcare Devices. *IEEE Access* **2022**, *10*, 60486–60495. [CrossRef]
2. Zhang, Z.; Mahmud, M.A.P.; Kouzani, A.Z. FitNN: A Low-Resource FPGA-Based CNN Accelerator for Drones. *IEEE Internet Things J.* **2022**, *9*, 21357–21369. [CrossRef]
3. Li, X.; Gong, X.; Wang, D.; Zhang, J.; Baker, T.; Zhou, J.; Lu, T. ABM-SpConv-SIMD: Accelerating Convolutional Neural Network Inference for Industrial IoT Applications on Edge Devices. *IEEE Trans. Netw. Sci. Eng.* **2023**, *10*, 3071–3085. [CrossRef]
4. Nikouei, S.Y.; Chen, Y.; Song, S.; Xu, R.; Choi, B.Y.; Faughnan, T.R. Smart Surveillance as an Edge Network Service: From Harr-Cascade, SVM to a Lightweight CNN. *arXiv* **2018**, arXiv:1805.00331. [CrossRef]
5. Tamimi, S.; Ebrahimi, Z.; Khaleghi, B.; Asadi, H. An Efficient SRAM-Based Reconfigurable Architecture for Embedded Processors. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* **2019**, *38*, 466–479. [CrossRef]
6. Wu, X.; Ma, Y.; Wang, M.; Wang, Z. A Flexible and Efficient FPGA Accelerator for Various Large-Scale and Lightweight CNNs. *IEEE Trans. Circuits Syst. Regul. Pap.* **2022**, *69*, 1185–1198. [CrossRef]
7. Irmak, H.; Ziener, D.; Alachiotis, N. Increasing Flexibility of FPGA-based CNN Accelerators with Dynamic Partial Reconfiguration. In Proceedings of the 2021 31st International Conference on Field-Programmable Logic and Applications (FPL), Dresden, Germany, 30 August–3 September 2021; pp. 306–311. [CrossRef]
8. Wei, Z.; Arora, A.; Li, R.; John, L. HLSDataset: Open-Source Dataset for ML-Assisted FPGA Design using High Level Synthesis. In Proceedings of the 2023 IEEE 34th International Conference on Application-specific Systems, Architectures and Processors (ASAP), Porto, Portugal, 19–21 July 2023; pp. 197–204. [CrossRef]
9. Mohammadi Makrani, H.; Farahmand, F.; Sayadi, H.; Bondi, S.; Pudukotai Dinakarrao, S.M.; Homayoun, H.; Rafatirad, S. Pyramid: Machine Learning Framework to Estimate the Optimal Timing and Resource Usage of a High-Level Synthesis Design. In Proceedings of the 2019 29th International Conference on Field Programmable Logic and Applications (FPL), Barcelona, Spain, 8–12 September 2019; pp. 397–403. [CrossRef]
10. Ullah, S.; Rehman, S.; Shafique, M.; Kumar, A. High-Performance Accurate and Approximate Multipliers for FPGA-Based Hardware Accelerators. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* **2022**, *41*, 211–224. [CrossRef]
11. Li, S.; Luo, Y.; Sun, K.; Yadav, N.; Choi, K.K. A Novel FPGA Accelerator Design for Real-Time and Ultra-Low Power Deep Convolutional Neural Networks Compared with Titan X GPU. *IEEE Access* **2020**, *8*, 105455–105471. [CrossRef]
12. Yang, C.; Wang, Y.; Zhang, H.; Wang, X.; Geng, L. A Reconfigurable CNN Accelerator using Tile-by-Tile Computing and Dynamic Adaptive Data Truncation. In Proceedings of the 2019 IEEE International Conference on Integrated Circuits, Technologies and Applications (ICTA), Chengdu, China, 13–15 November 2019; pp. 73–74. [CrossRef]

13. Zhang, X.; Ma, Y.; Xiong, J.; Hwu, W.M.W.; Kindratenko, V.; Chen, D. Exploring HW/SW Co-Design for Video Analysis on CPU-FPGA Heterogeneous Systems. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* **2022**, *41*, 1606–1619. [CrossRef]
14. Tensil. Learn Tensil with ResNet and PYNQ Z1. Available online: https://k155la3.blog/2022/04/04/tensil-tutorial-for-yolo-v4-tiny-on-ultra96-v2/ (accessed on 15 December 2022).
15. Kim, Y.; Tong, Q.; Choi, K.; Lee, E.; Jang, S.J.; Choi, B.H. System Level Power Reduction for YOLO2 Sub-modules for Object Detection of Future Autonomous Vehicles. In Proceedings of the 2018 International SoC Design Conference (ISOCC), Daegu, Republic of Korea, 12–15 November 2018; pp. 151–155. [CrossRef]
16. Kim, Y.; Kim, H.; Yadav, N.; Li, S.; Choi, K.K. Low-Power RTL Code Generation for Advanced CNN Algorithms toward Object Detection in Autonomous Vehicles. *Electronics* **2020**, *9*, 478. [CrossRef]
17. Kim, H.; Choi, K. Low Power FPGA-SoC Design Techniques for CNN-based Object Detection Accelerator. In Proceedings of the 2019 IEEE 10th Annual Ubiquitous Computing, Electronics and Mobile Communication Conference (UEMCON), New York, NY, USA, 10–12 October 2019; pp. 1130–1134. [CrossRef]
18. Kim, V.H.; Choi, K.K. A Reconfigurable CNN-Based Accelerator Design for Fast and Energy-Efficient Object Detection System on Mobile FPGA. *IEEE Access* **2023**, *11*, 59438–59445. [CrossRef]
19. Zhang, Y.; Tong, Q.; Li, L.; Wang, W.; Choi, K.; Jang, J.; Jung, H.; Ahn, S.Y. Automatic Register Transfer level CAD tool design for advanced clock gating and low power schemes. In Proceedings of the 2012 International SoC Design Conference (ISOCC), Jeju Island, Republic of Korea, 4–7 November 2012; pp. 21–24. [CrossRef]
20. Advanced Micro Devices, Inc. AMD ZCU104. Available online: https://www.amd.com/en/products/adaptive-socs-and-fpgas/evaluation-boards/zcu104.html (accessed on 19 June 2024).
21. Nguyen, D.-D.; Nguyen, D.-T.; Le, M.-T.; Nguyen, Q.-C. FPGA-SoC implementation of YOLOv4 for flying-object detection. *J. Real-Time Image Process.* **2024**, *21*, *63*. [CrossRef]
22. Valadanzoj, Z.; Daryanavard, H.; Harifi, A. High-speed YOLOv4-tiny hardware accelerator for self-driving automotive. *J. Supercomput.* **2024**, *80*, 6699–6724. [CrossRef]
23. Wang, Y.; Liao, Y.; Yang, J.; Wang, H.; Zhao, Y.; Zhang, C.; Xiao, B.; Xu, F.; Gao, Y.; Xu, M.; Zheng, J. An FPGA-based online reconfigurable CNN edge computing device for object detection. *Microelectron. J.* 2023, *137*, 105805. [CrossRef]
24. Heller, D.; Rizk, M.; Douguet, R.; Baghdadi, A.; Diguet, J.-P. Marine objects detection using deep learning on embedded edge devices. In Proceedings of the IEEE International Workshop on Rapid System Prototyping (RSP), Shanghai, China, 13 October 2022; pp. 1–7. Available online: https://ieeexplore.ieee.org/document/10039025 (accessed on 17 August 2025).
25. Montgomerie-Corcoran, A.; Toupas, P.; Yu, Z.; Bouganis, C.-S. SATAY: A streaming architecture toolflow for accelerating YOLO models on FPGA devices. In Proceedings of the International Conference on Field Programmable Technology (ICFPT), Yokohama, Japan, 11–14 December 2023; pp. 179–187. Available online: https://ieeexplore.ieee.org/document/10416135 (accessed on 17 August 2025).
26. Amin, R.A.; Hasan, M.; Wiese, V.; Obermaisser, R. FPGA-Based Real-Time Object Detection and Classification System Using YOLO for Edge Computing. In Proceedings of the IEEE International Conference on Consumer Electronics (ICCE), Las Vegas, NV, USA, 5–8 January 2024; pp. 1–6. Available online: https://ieeexplore.ieee.org/document/10537163/references#references (accessed on 17 August 2025).